

تمرین ششم مدرسه تابستانه تحلیل داده

در تمرین آخر مدرسه تابستانه تحلیل داده ما سعی می کنیم که با استفاده از دیتابیس که در : تمارین قبل ساختیم

1. دیتابیس را به پایتون متصل کنیم
2. به ۱۸ سوال مربوط به داده ها پاسخ دهیم

وارد کردن کتابخانه های مورد نیاز

- numpy: ریاضی
- pandas: برای تحلیل و کار کردن با دیتافریم ها
- matplotlib: رسم کردن نمودار
- mysql.connector: برای متصل کردن دیتابیس به پایتون

چون من میخوام که کد خودم را در گیتهاب آپلود کنم و از درز کردن پسورد جلوگیری کنم از یک فایل برای قرار دادن نام کاربری و رمز .. استفاده کردم و بعد از آن فایل فراخوانی کردم

برای انجام این کار از کتابخانه

dotenv

استفاده کردم

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import mysql.connector

from dotenv import load_dotenv
import os

load_dotenv()

True
```

1.

برای ساخت یک محیط مجازی از

```
python3 -m venv dataAnalytics
```

استفاده کردم چون در گذشته تجربه خوبی با استفاده از

Anaconda

نداشتم

به دلیل اینکه نیاز بود تمام خروجی ها کد ها را در گزارش می آوردم با استفاده از نوت بوک در پایتون این کار به سادگی قابل انجام می شود.

```
conn = mysql.connector.connect(  
    host="127.0.0.1",  
    port=3306,  
    user = os.getenv('DB_USER'),  
    password = os.getenv('DB_PASSWORD')  
)  
  
df = pd.read_sql("SELECT * FROM KohanNegar.order_detail2", conn)  
print(df.head())  
print('- '*50)  
print(df.head(10))  
print('- '*50)  
print(df.tail())
```

	order_id	detail_id	product_id	quantity	price
0	1	1	104	1	19370000
1	2	3	120	1	30670000
2	3	8	116	2	23690000
3	4	10	140	1	30000000
4	5	11	3	1	13010000

	order_id	detail_id	product_id	quantity	price
0	1	1	104	1	19370000
1	2	3	120	1	30670000
2	3	8	116	2	23690000
3	4	10	140	1	30000000
4	5	11	3	1	13010000
5	6	13	132	2	24050000
6	7	14	7	1	4610000
7	8	19	9	1	11130000
8	9	20	2	1	11100000
9	10	21	48	1	4640000

	order_id	detail_id	product_id	quantity	price
5671	5672	13629	16	1	11670000
5672	5673	13632	7	1	4610000
5673	5674	13634	8	1	7370000
5674	5675	13635	121	1	25290000
5675	5676	13636	181	1	64300000

```
/var/folders/h1/gmdl87ks4xqg3lg3hddyxr4r0000gn/T/  
ipykernel_67858/3326034561.py:1: UserWarning: pandas only supports  
SQLAlchemy connectable (engine/connection) or database string URI or  
sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please  
consider using SQLAlchemy.
```

```
df = pd.read_sql("SELECT * FROM KohanNegar.order_detail2", conn)
```

2.

تعداد سطرها و ستون های دیتافریم

```
print(df.shape)
print(df.columns)

(5676, 5)
Index(['order_id', 'detail_id', 'product_id', 'quantity', 'price'],
      dtype='str')

print(df.info)

<bound method DataFrame.info of          order_id  detail_id  product_id
quantity      price
0              1          1         104          1  19370000
1              2          3         120          1  30670000
2              3          8         116          2  23690000
3              4         10         140          1  30000000
4              5         11          3          1  13010000
...          ...          ...          ...          ...
5671          5672      13629          16          1  11670000
5672          5673      13632           7          1   4610000
5673          5674      13634           8          1   7370000
5674          5675      13635          121          1  25290000
5675          5676      13636          181          1  64300000

[5676 rows x 5 columns]>
```

3. تحلیل آماری مقدماتی

```
print(df.describe)

<bound method NDFrame.describe of          order_id  detail_id
product_id  quantity      price
0              1          1         104          1  19370000
1              2          3         120          1  30670000
2              3          8         116          2  23690000
3              4         10         140          1  30000000
4              5         11          3          1  13010000
...          ...          ...          ...          ...
5671          5672      13629          16          1  11670000
5672          5673      13632           7          1   4610000
5673          5674      13634           8          1   7370000
5674          5675      13635          121          1  25290000
5675          5676      13636          181          1  64300000

[5676 rows x 5 columns]>

print(df['quantity'].describe())
```

```
count      5676.000000
mean        1.252290
std         0.649055
min         1.000000
25%         1.000000
50%         1.000000
75%         1.000000
max         5.000000
Name: quantity, dtype: float64
```

4. پاکسازی داده ها

در اینجا به دلیل اینکه هیچ داده از دست رفته ای نداشتیم و هیچ داده تکراری نیز وجود نداشت از کدهایی برای حذف این داده ها استفاده نکردیم.

```
print(df.isnull().sum())

order_id      0
detail_id     0
product_id    0
quantity      0
price         0
dtype: int64
```

5. پیدا کردن دادهای تاهنجار

```
df.duplicated().sum()

np.int64(0)
```

6. فرمت دیتا

در پایگاه داده من تاریخ را به زمانی تعریف کرده بودم به همین دلیل مشکل از این بابت وجود نداشت.

```
df = pd.read_sql("SELECT * FROM KohanNegar.orders", conn)
print(df['order_date'].dtype)

datetime64[us]

/var/folders/h1/gmdl87ks4xqg31g3hddyxr4r0000gn/T/
ipykernel_67858/812198704.py:1: UserWarning: pandas only supports
SQLAlchemy connectable (engine/connection) or database string URI or
sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please
consider using SQLAlchemy.
  df = pd.read_sql("SELECT * FROM KohanNegar.orders", conn)

print(df['order_date'].dt.year)
```

```

0      2026
1      2025
2      2026
3      2026
4      2026
...
5671   2024
5672   2025
5673   2023
5674   2024
5675   2022
Name: order_date, Length: 5676, dtype: int32

```

```
print(df['order_date'].dt.month)
```

```

0      4
1      9
2      6
3      5
4      8
...
5671   3
5672   5
5673   3
5674   1
5675  12
Name: order_date, Length: 5676, dtype: int32

```

7. ایجاد ستون جدید میزان فروش.

```

df = pd.read_sql("SELECT * FROM KohanNegar.order_detail2", conn)
df['sales'] = df['quantity'] * df['price']
print(df)

```

	order_id	detail_id	product_id	quantity	price	sales
0	1	1	104	1	19370000	19370000
1	2	3	120	1	30670000	30670000
2	3	8	116	2	23690000	47380000
3	4	10	140	1	30000000	30000000
4	5	11	3	1	13010000	13010000
...	...	...	...	...	...	...
5671	5672	13629	16	1	11670000	11670000
5672	5673	13632	7	1	4610000	4610000
5673	5674	13634	8	1	7370000	7370000
5674	5675	13635	121	1	25290000	25290000
5675	5676	13636	181	1	64300000	64300000

```
[5676 rows x 6 columns]
```

```
/var/folders/h1/gmdl87ks4xqg3lg3hddyxr4r0000gn/T/ipykernel_67858/2667861078.py:1: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.  
df = pd.read_sql("SELECT * FROM KohanNegar.order_detail2", conn)
```

9. استفاده از نامیای

```
average_sales = np.mean(df['sales'])  
print(average_sales)  
print('- '*100)  
max_sales = np.max(df['sales'])  
print(max_sales)  
print('- '*100)  
min_sales = np.min(df['sales'])  
print(min_sales)
```

44799906.62438337

870450000

2020000

10. تحلیل فروش

- جمع فروش : 254284270000
- میانگین فروش : 44799906.62438337
- بیشترین مقدار فروش: 870450000

```
total_sales = df['sales'].sum()  
print(f' جمع فروش : {total_sales}')
```

254284270000 : جمع فروش

```
mean_sales = df['sales'].mean()  
print(f' میانگین فروش : {mean_sales}')
```

44799906.62438337 : میانگین فروش

```
maximum_sales = df['sales'].max()  
print(f' بیشترین مقدار فروش : {maximum_sales}')
```

870450000 : بیشترین مقدار فروش

فروش بر اساس محصول

```
product_sales = df.groupby('product_id')['sales'].sum()
product_sales = product_sales.sort_values(ascending=False)
print('product sales:', product_sales)
```

```
product sales: product_id
153      27332130000
149      18546250000
155      13003050000
147       7949660000
188      6569640000
...
25       274300000
74       264500000
68       229200000
78       144900000
66       105000000
Name: sales, Length: 195, dtype: int64
```

فروش بر اساس شهر

```
users = pd.read_sql("SELECT users.id as 'user_id',
users.user_name,users.city FROM KohanNegar.users", conn)
orders = pd.read_sql("SELECT orders.id as 'order_id', orders.user_id
FROM KohanNegar.orders", conn)
```

```
/var/folders/h1/gmdl87ks4xqg3lg3hddyxr4r0000gn/T/
ipykernel_67858/3572892387.py:1: UserWarning: pandas only supports
SQLAlchemy connectable (engine/connection) or database string URI or
sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please
consider using SQLAlchemy.
```

```
users = pd.read_sql("SELECT users.id as 'user_id',
users.user_name,users.city FROM KohanNegar.users", conn)
/var/folders/h1/gmdl87ks4xqg3lg3hddyxr4r0000gn/T/ipykernel_67858/35728
92387.py:2: UserWarning: pandas only supports SQLAlchemy connectable
(engine/connection) or database string URI or sqlite3 DBAPI2
connection. Other DBAPI2 objects are not tested. Please consider using
SQLAlchemy.
```

```
orders = pd.read_sql("SELECT orders.id as 'order_id', orders.user_id
FROM KohanNegar.orders", conn)
```

```
result_1 = pd.merge(orders, users, left_on = 'user_id',
right_on='user_id', how='inner')
result_1
```

	order_id	user_id	user_name	city
0	36	10	hamidreza.afshar	Rasht
1	37	10	hamidreza.afshar	Rasht
2	38	10	hamidreza.afshar	Rasht
3	39	10	hamidreza.afshar	Rasht

4	40	10	hamidreza.afshar		Rasht
...	...	...	...		...
5671	3944	997	navid.eshghi	Bandar	Anzali
5672	3945	997	navid.eshghi	Bandar	Anzali
5673	3946	997	navid.eshghi	Bandar	Anzali
5674	3947	997	navid.eshghi	Bandar	Anzali
5675	3948	998	kamran.moradi		Rasht

[5676 rows x 4 columns]

```
result_2 = pd.read_sql("SELECT order_id, total_amount FROM
KohanNegar.payments", conn)
result_2
```

/var/folders/h1/gmdl87ks4xqg31g3hddyxr4r0000gn/T/
ipykernel_67858/1507164265.py:1: UserWarning: pandas only supports
SQLAlchemy connectable (engine/connection) or database string URI or
sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please
consider using SQLAlchemy.

```
result_2 = pd.read_sql("SELECT order_id, total_amount FROM
KohanNegar.payments", conn)
```

	order_id	total_amount
0	1	49370000
1	2	90930000
2	4	27000000
3	5	18680000
4	6	40885000
...	...	...
3696	5671	216810000
3697	5673	7785000
3698	5674	7370000
3699	5675	25290000
3700	5676	64300000

[3701 rows x 2 columns]

```
result_3 = pd.merge(result_1, result_2, on='order_id', how='inner')
result_3
```

	order_id	user_id	user_name	city	total_amount
0	36	10	hamidreza.afshar	Rasht	19958000
1	37	10	hamidreza.afshar	Rasht	39980000
2	39	10	hamidreza.afshar	Rasht	5670000
3	40	10	hamidreza.afshar	Rasht	69090000
4	41	10	hamidreza.afshar	Rasht	20077000
...	...	...	...	...	...
3696	3944	997	navid.eshghi	Bandar Anzali	197980000
3697	3945	997	navid.eshghi	Bandar Anzali	51880000
3698	3946	997	navid.eshghi	Bandar Anzali	78552000
3699	3947	997	navid.eshghi	Bandar Anzali	516170000

```
3700      3948      998      kamran.moradi      Rasht      13090000
```

```
[3701 rows x 5 columns]
```

```
city_sales = result_3.groupby('city')['total_amount'].sum()
city_sales = city_sales.sort_values(ascending=False)
city_sales
```

```
city
Rasht      210785364000
Nowshahr    95509980000
Bandar Anzali 72328970000
Name: total_amount, dtype: int64
```

13. تعداد فروش هر محصول

```
pd.read_sql()
```

```
result_1 = pd.read_sql("SELECT * FROM KohanNegar.orders WHERE was_paid = 1", conn)
result_1
```

```
/var/folders/h1/gmdl87ks4xqg31g3hddyxr4r0000gn/T/ipykernel_67858/47931387.py:1: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
```

```
result_1 = pd.read_sql("SELECT * FROM KohanNegar.orders WHERE was_paid = 1", conn)
```

	id	user_id	sales_channel	order_date	was_paid
0	1	2	online	2026-04-28 08:58:00	1
1	2	2	in_person	2025-09-30 16:22:00	1
2	3	3	online	2026-06-24 10:17:00	1
3	4	3	online	2026-05-27 09:23:00	1
4	5	3	online	2026-08-29 14:59:00	1
...	...	...	...	...	...
5229	5672	1500	online	2024-03-19 19:56:00	1
5230	5673	1500	online	2025-05-01 14:26:00	1
5231	5674	1500	in_person	2023-03-02 13:06:00	1
5232	5675	1500	in_person	2024-01-12 16:06:00	1
5233	5676	1500	online	2022-12-22 20:03:00	1

```
[5234 rows x 5 columns]
```

```
result_2 = pd.read_sql("SELECT * FROM KohanNegar.order_detail2", conn)
result_2
```

```
/var/folders/h1/gmdl87ks4xqg31g3hddyxr4r0000gn/T/ipykernel_67858/1973391338.py:1: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
```

```
result_2 = pd.read_sql("SELECT * FROM KohanNegar.order_detail2", conn)
```

	order_id	detail_id	product_id	quantity	price
0	1	1	104	1	19370000
1	2	3	120	1	30670000
2	3	8	116	2	23690000
3	4	10	140	1	30000000
4	5	11	3	1	13010000
...	...	...	...	...	...
5671	5672	13629	16	1	11670000
5672	5673	13632	7	1	4610000
5673	5674	13634	8	1	7370000
5674	5675	13635	121	1	25290000
5675	5676	13636	181	1	64300000

```
[5676 rows x 5 columns]
```

```
join_1 = pd.merge(result_1, result_2, left_on = 'id',  
right_on='order_id', how='inner')  
join_1
```

	id	user_id	sales_channel	order_date	was_paid
order_id \					
0	1	2	online	2026-04-28 08:58:00	1
1					
1	2	2	in_person	2025-09-30 16:22:00	1
2					
2	3	3	online	2026-06-24 10:17:00	1
3					
3	4	3	online	2026-05-27 09:23:00	1
4					
4	5	3	online	2026-08-29 14:59:00	1
5					
...	...	...	...	...	...
..					
5229	5672	1500	online	2024-03-19 19:56:00	1
5672					
5230	5673	1500	online	2025-05-01 14:26:00	1
5673					
5231	5674	1500	in_person	2023-03-02 13:06:00	1
5674					
5232	5675	1500	in_person	2024-01-12 16:06:00	1
5675					
5233	5676	1500	online	2022-12-22 20:03:00	1

5676

	detail_id	product_id	quantity	price
0	1	104	1	19370000
1	3	120	1	30670000
2	8	116	2	23690000
3	10	140	1	30000000
4	11	3	1	13010000
...	...	...	...	...
5229	13629	16	1	11670000
5230	13632	7	1	4610000
5231	13634	8	1	7370000
5232	13635	121	1	25290000
5233	13636	181	1	64300000

[5234 rows x 10 columns]

```
result_3 = pd.read_sql("SELECT * FROM KohanNegar.products", conn)
result_3
```

/var/folders/h1/gmdl87ks4xqg31g3hddyxr4r0000gn/T/

ipykernel_67858/3838753790.py:1: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
result_3 = pd.read_sql("SELECT * FROM KohanNegar.products", conn)
```

	id	title \
0	1	Imperial Classical Verses Vol. 4
1	2	Coastal Classical Masters Vol. 65
2	3	Golden Classical Symphony Vol. 26
3	4	Vintage Classical Legacy Vol. 70
4	5	Coastal Classical Treasures Vol. 76
..	...	...
190	191	Mystic Numbered Masters Vol. 84
191	192	Vintage Numbered Verses Vol. 97
192	193	Coastal Numbered Masters Vol. 3
193	194	Eternal Numbered Masters Vol. 17
194	195	Vintage Numbered Chronicles Vol. 73

	category_id	detail	price
0	7	A fine collection of classical persian poetry ...	2570000
1	7	A fine collection of classical persian poetry ...	11100000
2	7	A fine collection of classical persian poetry ...	13010000
3	7	A fine collection of classical persian poetry ...	8040000

```

4   A fine collection of classical persian poetry ...    5670000
7
..
...
190 A fine collection of numbered collector editio... 38420000
29
191 A fine collection of numbered collector editio... 28330000
29
192 A fine collection of numbered collector editio... 186790000
29
193 A fine collection of numbered collector editio... 147120000
29
194 A fine collection of numbered collector editio... 78510000
29

```

```
[195 rows x 5 columns]
```

```

result_4 = pd.read_sql("SELECT * FROM KohanNegar.categories",conn)
result_4

```

```

/var/folders/h1/gmdl87ks4xqg31g3hddyxr4r0000gn/T/
ipykernel_67858/2278804333.py:1: UserWarning: pandas only supports
SQLAlchemy connectable (engine/connection) or database string URI or
sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please
consider using SQLAlchemy.

```

```
result_4 = pd.read_sql("SELECT * FROM KohanNegar.categories",conn)
```

	id	title	parent_id
0	7	Classical Persian Poetry	1
1	8	Contemporary Poetry	1
2	9	Translated World Poetry	1
3	10	Illustrated Poetry Editions	1
4	11	Ancient Persia	2
5	12	Islamic Era History	2
6	13	Modern Iranian History	2
7	14	World History	2
8	15	Local/Regional History	2
9	16	Miniature Reproductions	3
10	17	Oil Painting Books	3
11	18	Watercolor Collections	3
12	19	Calligraphy Art	3
13	20	Black & White Photography	4
14	21	Landscape Photography	4
15	22	Portrait Photography	4
16	23	Documentary Photography	4
17	24	Single-Artist Retrospectives	5
18	25	Limited Edition Artist Sets	5
19	26	Signed Artist Collections	5
20	27	Antique Reprints	6

21	28	Facsimile Editions	6
22	29	Numbered Collector Editions	6

```
join_2 = pd.merge(result_3, result_4, left_on = 'category_id',
right_on='id', how='inner')
join_2
```

	id_x	title_x \
0	1	Imperial Classical Verses Vol. 4
1	2	Coastal Classical Masters Vol. 65
2	3	Golden Classical Symphony Vol. 26
3	4	Vintage Classical Legacy Vol. 70
4	5	Coastal Classical Treasures Vol. 76
..	...	...
190	191	Mystic Numbered Masters Vol. 84
191	192	Vintage Numbered Verses Vol. 97
192	193	Coastal Numbered Masters Vol. 3
193	194	Eternal Numbered Masters Vol. 17
194	195	Vintage Numbered Chronicles Vol. 73

	detail	price \
0	A fine collection of classical persian poetry ...	2570000
1	A fine collection of classical persian poetry ...	11100000
2	A fine collection of classical persian poetry ...	13010000
3	A fine collection of classical persian poetry ...	8040000
4	A fine collection of classical persian poetry ...	5670000
..	...	...
190	A fine collection of numbered collector editio...	38420000
191	A fine collection of numbered collector editio...	28330000
192	A fine collection of numbered collector editio...	186790000
193	A fine collection of numbered collector editio...	147120000
194	A fine collection of numbered collector editio...	78510000

	category_id	id_y	title_y	parent_id
0	7	7	Classical Persian Poetry	1
1	7	7	Classical Persian Poetry	1
2	7	7	Classical Persian Poetry	1
3	7	7	Classical Persian Poetry	1
4	7	7	Classical Persian Poetry	1
..	...	...	...	...
190	29	29	Numbered Collector Editions	6
191	29	29	Numbered Collector Editions	6
192	29	29	Numbered Collector Editions	6
193	29	29	Numbered Collector Editions	6
194	29	29	Numbered Collector Editions	6

[195 rows x 8 columns]

```
join_3 = pd.merge(join_1, join_2, left_on = 'id', right_on='id_x',
how='left')
join_3
```

	id	user_id	sales_channel	order_date	was_paid
order_id \					
0	1	2	online	2026-04-28 08:58:00	1
1					
1	2	2	in_person	2025-09-30 16:22:00	1
2					
2	3	3	online	2026-06-24 10:17:00	1
3					
3	4	3	online	2026-05-27 09:23:00	1
4					
4	5	3	online	2026-08-29 14:59:00	1
5					
...	...	...	...	...	...
...					
5229	5672	1500	online	2024-03-19 19:56:00	1
5672					
5230	5673	1500	online	2025-05-01 14:26:00	1
5673					
5231	5674	1500	in_person	2023-03-02 13:06:00	1
5674					
5232	5675	1500	in_person	2024-01-12 16:06:00	1
5675					
5233	5676	1500	online	2022-12-22 20:03:00	1
5676					

	detail_id	product_id	quantity	price_x	id_x \
0	1	104	1	19370000	1.0
1	3	120	1	30670000	2.0
2	8	116	2	23690000	3.0
3	10	140	1	30000000	4.0
4	11	3	1	13010000	5.0
...	...	...	...	...	...
5229	13629	16	1	11670000	NaN
5230	13632	7	1	4610000	NaN
5231	13634	8	1	7370000	NaN
5232	13635	121	1	25290000	NaN
5233	13636	181	1	64300000	NaN

	title_x \
0	Imperial Classical Verses Vol. 4
1	Coastal Classical Masters Vol. 65
2	Golden Classical Symphony Vol. 26
3	Vintage Classical Legacy Vol. 70
4	Coastal Classical Treasures Vol. 76
...	...
5229	NaN

5230	NaN
5231	NaN
5232	NaN
5233	NaN

		detail	price_y \
0	A fine collection of classical persian poetry ...		2570000.0
1	A fine collection of classical persian poetry ...		11100000.0
2	A fine collection of classical persian poetry ...		13010000.0
3	A fine collection of classical persian poetry ...		8040000.0
4	A fine collection of classical persian poetry ...		5670000.0
...		...	...
5229		NaN	NaN
5230		NaN	NaN
5231		NaN	NaN
5232		NaN	NaN
5233		NaN	NaN

	category_id	id_y	title_y	parent_id
0	7.0	7.0	Classical Persian Poetry	1.0
1	7.0	7.0	Classical Persian Poetry	1.0
2	7.0	7.0	Classical Persian Poetry	1.0
3	7.0	7.0	Classical Persian Poetry	1.0
4	7.0	7.0	Classical Persian Poetry	1.0
...	...	...	...	...
5229	NaN	NaN	NaN	NaN
5230	NaN	NaN	NaN	NaN
5231	NaN	NaN	NaN	NaN
5232	NaN	NaN	NaN	NaN
5233	NaN	NaN	NaN	NaN

[5234 rows x 18 columns]

```
product_counts = (
    join_3.groupby('title_y')['quantity'].sum()
    .sort_values(ascending=False)
)
```

product_counts

title_y	
Classical Persian Poetry	15
Landscape Photography	14
Single-Artist Retrospectives	13
Contemporary Poetry	13
Ancient Persia	12
Illustrated Poetry Editions	12
Islamic Era History	11
Limited Edition Artist Sets	11
World History	11

Facsimile Editions	10
Documentary Photography	10
Local/Regional History	10
Numbered Collector Editions	9
Black & White Photography	9
Watercolor Collections	9
Calligraphy Art	8
Modern Iranian History	7
Signed Artist Collections	7
Antique Reprints	6
Miniature Reproductions	6
Portrait Photography	6
Translated World Poetry	6
Oil Painting Books	5

Name: quantity, dtype: int64

تحليل فروش ماهانه.14

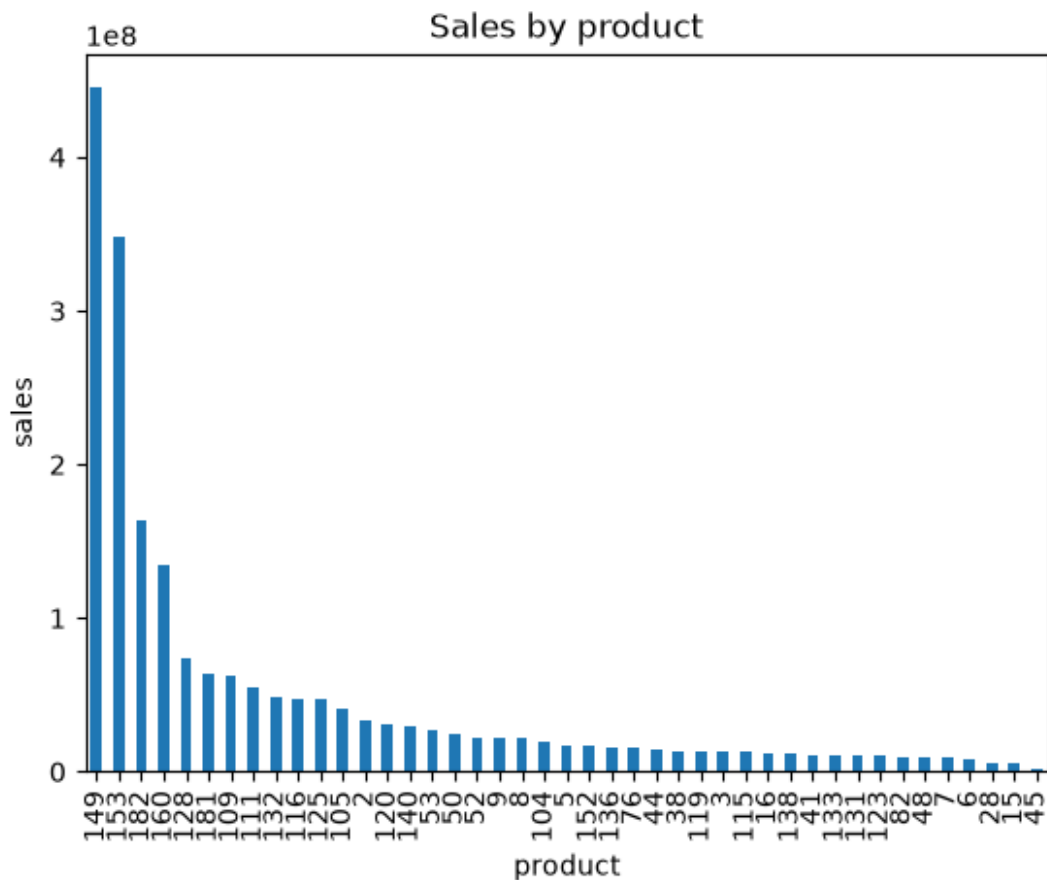
```
df = pd.read_sql("SELECT * FROM KohanNegar.payments", conn)
df['month'] = df['payment_date'].dt.month
monthly_sales = df.groupby("month")['total_amount'].sum()
monthly_sales = monthly_sales.sort_values(ascending=False)
print('monthly sales', monthly_sales)
```

```
monthly sales month
7      47285032000
6      37119915000
1      35812106000
2      33822696000
3      33558235000
12     32463167000
5      31493795000
11     31066000000
8      29544171000
9      28681395000
4      22538048000
10     15239754000
Name: total_amount, dtype: int64
```

```
/var/folders/h1/gmdl87ks4xqg3lg3hddyxr4r0000gn/T/
ipykernel_66032/4014613936.py:1: UserWarning: pandas only supports
SQLAlchemy connectable (engine/connection) or database string URI or
sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please
consider using SQLAlchemy.
  df = pd.read_sql("SELECT * FROM KohanNegar.payments", conn)
```

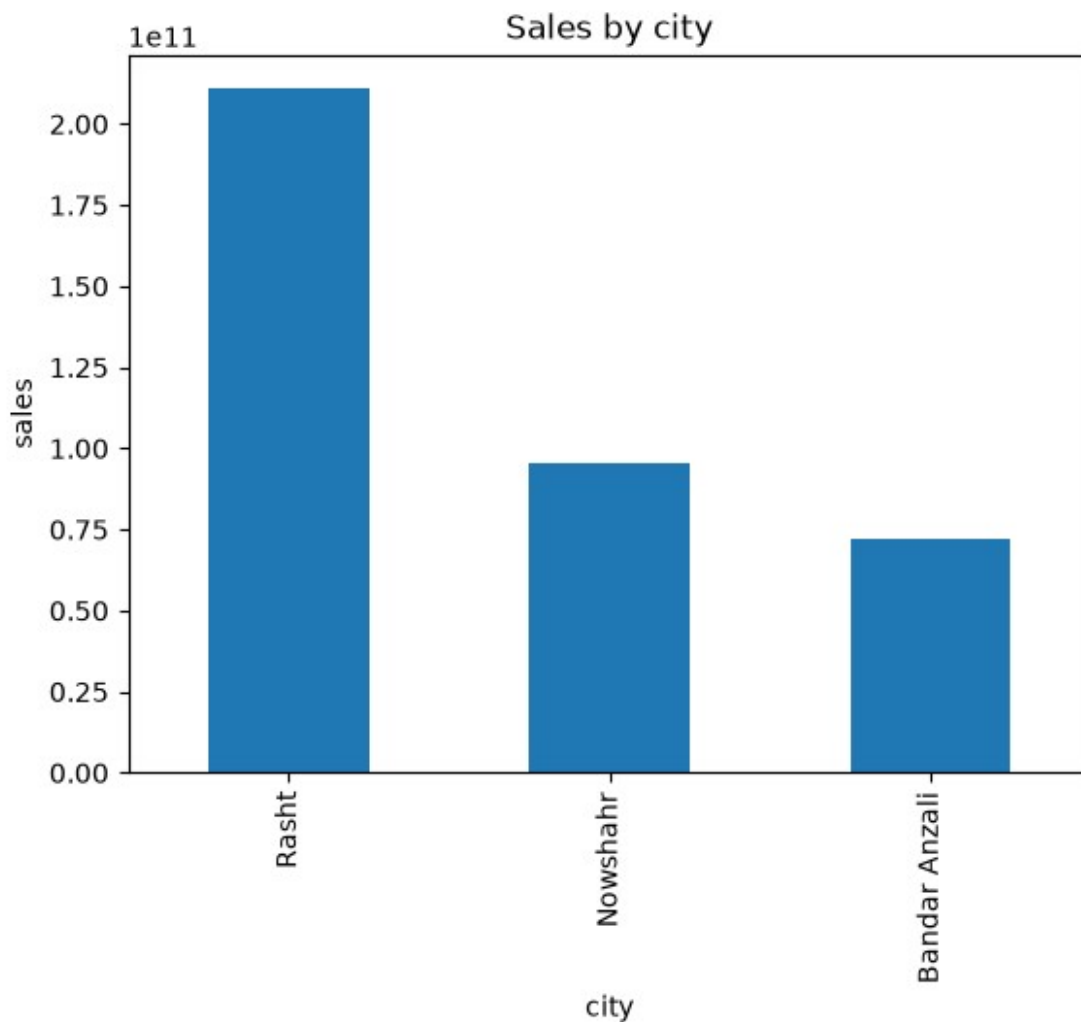
15. رسم نمودار

```
product_sales.plot(kind='bar')
plt.title('Sales by product')
plt.xlabel('product')
plt.ylabel('sales')
plt.show()
```



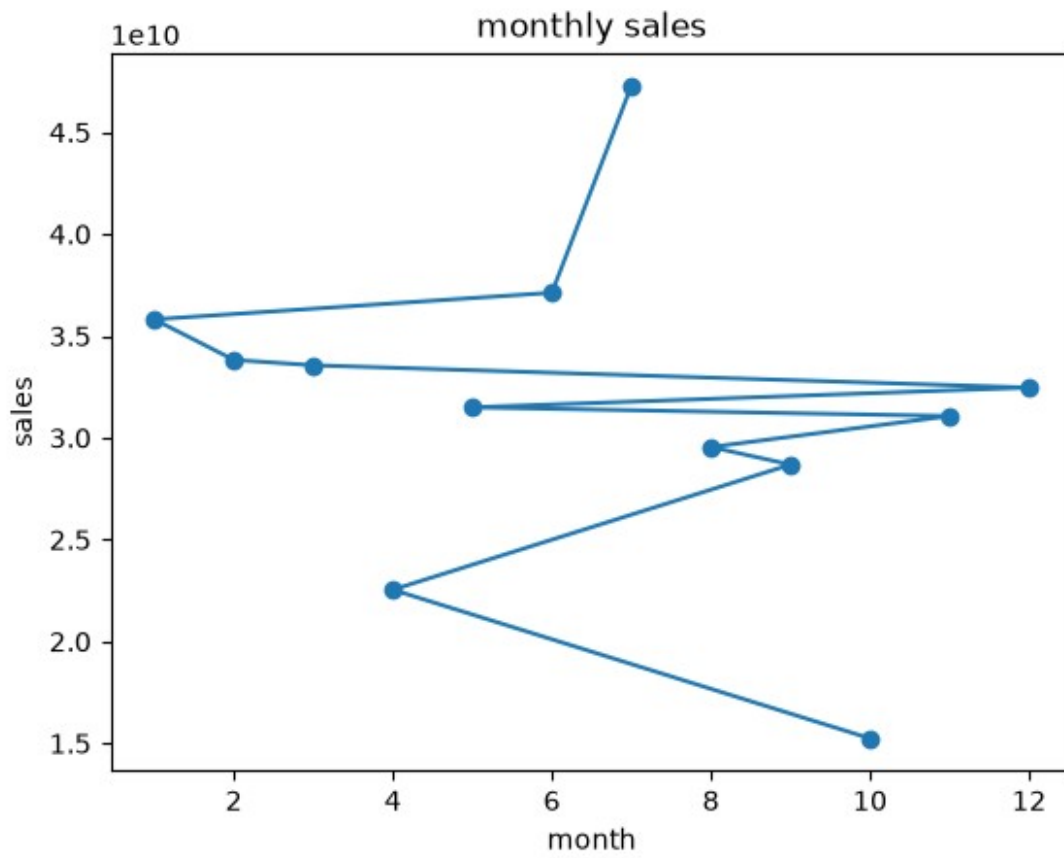
16. نمودار فروش هر شهر

```
city_sales.plot(kind = 'bar')
plt.title('Sales by city')
plt.xlabel('city')
plt.ylabel('sales')
plt.show()
```



17. فروش ماهانه نمودار خطی

```
monthly_sales.plot(kind='line', marker= 'o')  
plt.title('monthly sales')  
plt.xlabel('month')  
plt.ylabel('sales')  
plt.show()
```



18. نمودار دایره ای

```
monthly_sales.plot(kind='pie', autopct = '%1.1f%%')
plt.title('product sales share')
plt.show()
```

product sales share

